

Sorted Singly Linked List

Sorted Singly Linked List

Meld Operation

```
// Function to merge two linked lists (meld operation)
void meld(Node** list1, Node* list2) {
    Node* current1 = *list1; Node* current2 = list2;
    Node* prev = NULL;

    // Iterate through both lists
    while (current1 != NULL && current2 != NULL)
        if (current1->data <= current2->data) {
            prev = current1; current1 = current1->next;
        } else {
            Node* next2 = current2->next;
            current2->next = current1;
            if (prev != NULL) prev->next = current2;
            else *list1 = current2;
            prev = current2; current2 = next2;
        }

    // Append any remaining nodes from list2 to list1
    if (current1 == NULL && current2 != NULL)
        if (prev != NULL) prev->next = current2;
        else *list1 = current2;
}
```

Reverse a Singly Linked List

Reverse a Singly Linked List

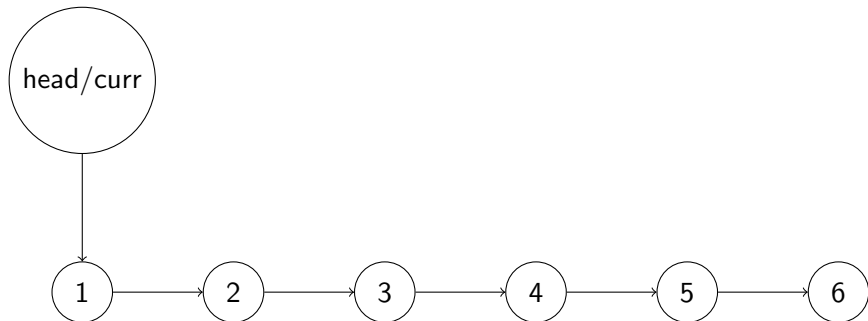
```
// Function to reverse a linked list
Node* reverseList(Node* head) {
    Node *prev = NULL;
    Node *curr = head;
    Node *next = NULL;

    while (curr != NULL) {
        next = curr->next;    // Store next node
        curr->next = prev;    // Reverse current node's pointer
        // Move pointers one position ahead
        prev = curr; curr = next;
    }

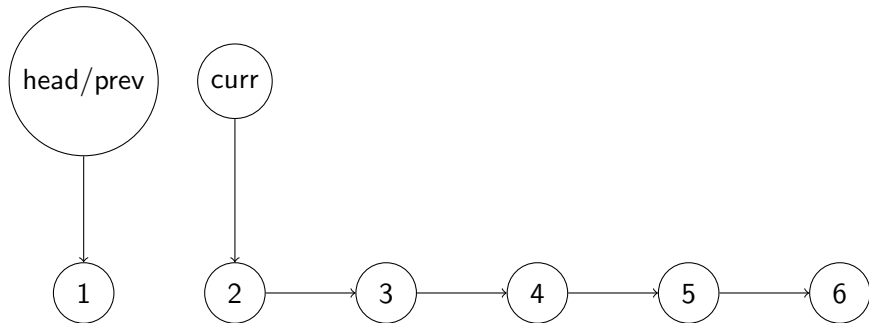
    // Update head to the last node which was previously the tail
    head = prev;

    return head;
}
```

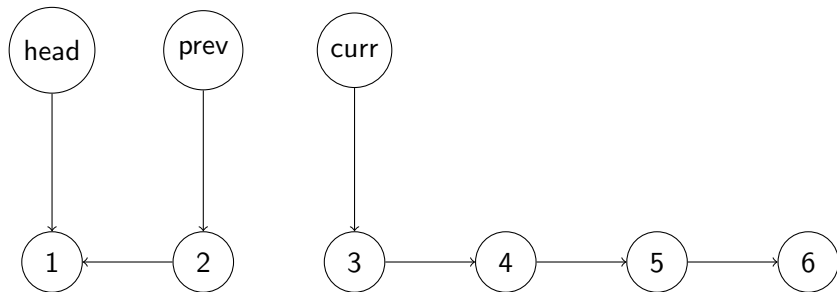
Initially $\text{curr}=\text{head}$, $\text{prev}=\text{NULL}$



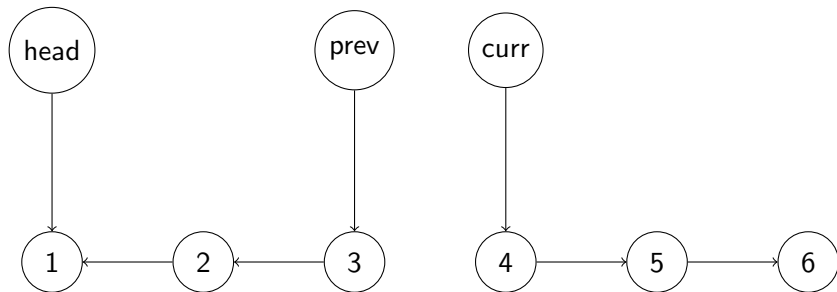
After 1 step



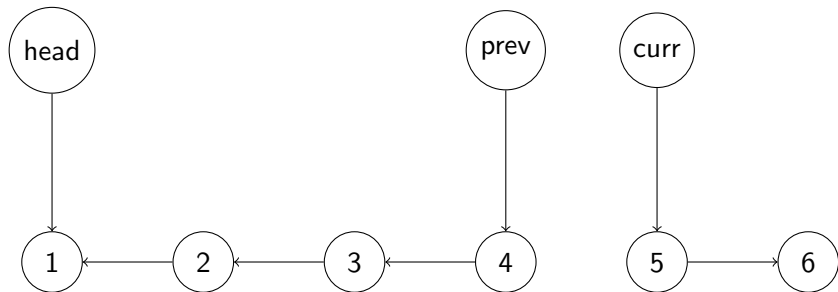
After 2 steps



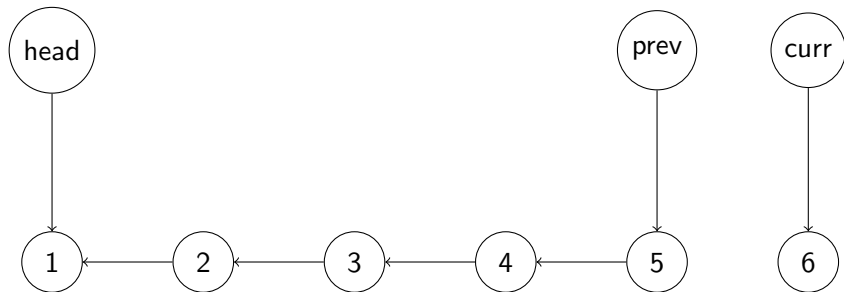
After 3 steps



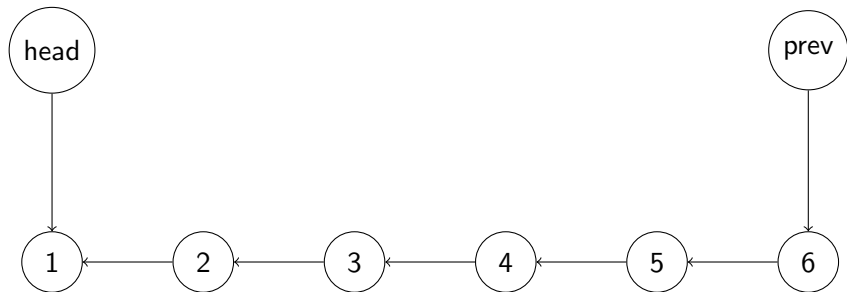
After 4 steps



After 5 steps



After 6 steps



The C Preprocessor

The C Preprocessor

File Inclusion

- ▶ File inclusion simplifies the handling of collections of `#defines` and declarations.
- ▶ Any source line of the form:
 - ▶ `#include "filename"`
 - ▶ `#include <filename>`is replaced by the contents of the file `filename`.
- ▶ If the filename is quoted, searching typically begins where the source program was found.
- ▶ If not found there, or if the name is enclosed in `<` and `>`, an implementation-defined rule is followed to find the file.
- ▶ An included file may itself contain `#include` lines.

The C Preprocessor

File Inclusion

- ▶ Commonly, several `#include` lines are used at the beginning of a source file.
- ▶ They include common `#define` statements and extern declarations.
- ▶ They also access function prototype declarations for library functions from headers like `<stdio.h>`.
- ▶ This is the preferred way to tie declarations together for a large program.
- ▶ It guarantees that all source files will be supplied with the same definitions and variable declarations, avoiding bugs.

The C Preprocessor

File Inclusion

- ▶ When an included file is changed, all files that depend on it must be recompiled.
- ▶ Header files need not be actual files; details are implementation-dependent.

The C Preprocessor

Macro Substitution

- ▶ A definition has the form
`#define name replacement-text.`
- ▶ Macro substitution replaces subsequent occurrences of the token `name` with the `replacement-text`.
- ▶ The scope of a name defined with `#define` is from its point of definition to the end of the source file.
- ▶ Definitions may use previous definitions, and substitutions are made only for tokens.
- ▶ Substitutions do not take place within quoted strings.

The C Preprocessor

Macro Substitution

```
#define forever for (;;)
/* infinite loop */
```

```
#define max(A, B) ((A) > (B) ? (A) : (B))
x = max(p+q, r+s);
/* Expands: x = ((p+q)>(r+s) ? (p+q) : (r+s)); */
/* Expressions evaluated twice; side effects! */
```

```
#define square(x) x * x      /* square(z+1) */
/* parentheses? order of evaluation issues */
```

The C Preprocessor

Macro Substitution

- ▶ Despite pitfalls, macros are valuable for performance and flexibility.
- ▶ Eg. from `<stdio.h>`: `getchar` and `putchar` defined as macros to avoid run-time overhead of function call per character processed.
- ▶ Names can be undefined with `#undef` to ensure consistency.

The C Preprocessor

Macro Substitution, # operator

```
#define dprint(expr) printf(#expr " = %g\n", expr)
```

```
dprint(x/y)
```

```
/* Expands into: printf("x/y" " = %g\n", x/y); */
```

```
/* Equivalent to: printf("x/y = %g\n", x/y); */
```

The C Preprocessor

Macro Substitution, ## operator

```
#define paste(front, back) front ## back
```

```
paste(name, 1)  
/* Expands into: name1 */
```

The C Preprocessor

Conditional Inclusion

- ▶ Conditional inclusion allows selective inclusion of code during compilation.
- ▶ `#if` evaluates a constant integer expression.
- ▶ If the expression is non-zero, subsequent lines until `#endif`, `#elif`, or `#else` are included.
- ▶ The expression `defined(name)` in a `#if` is 1 if the name has been defined, and 0 otherwise.

The C Preprocessor

Conditional Inclusion

```
#if !defined(HDR)
#define HDR
/* contents of hdr.h go here */
#endif
/* included only once */
```

The C Preprocessor

Conditional Inclusion

```
#if SYSTEM == SYSV
#define HDR "sysv.h"
#elif SYSTEM == BSD
#define HDR "bsd.h"
#elif SYSTEM == MSDOS
#define HDR "msdos.h"
#else
#define HDR "default.h"
#endif
#include HDR
```

The C Preprocessor

Conditional Inclusion

```
/* another way */  
#ifndef HDR  
#define HDR  
/* contents of hdr.h go here */  
#endif
```